



SwarmAssist

Multi-agent productivity & wellness swarm engine

A resilient AI productivity platform that orchestrates multiple intelligent agents to optimize daily schedules while preserving user wellness through adaptive planning and fault-tolerant AI inference.



5-agent AI swarm



Fault-tolerant inference



Wellness-aware planning

● Problem statement

Traditional productivity apps simply store tasks. Students and professionals need a system that actively reasons about them.



Unstructured task lists

No clear system to turn a messy list into an executable plan.



Poor prioritization

Important work gets buried under low-value tasks.



Energy crashes

Schedules ignore natural productivity and fatigue cycles.



No wellness integration

Breaks, hydration, and recovery are treated as an afterthought.



AI systems that freeze during slow local inference break trust during live demos and daily use alike.

● Solution & key highlights

A sequential multi-agent swarm engine where each agent owns a dedicated responsibility — coordinated asynchronously, with the UI protected by intelligent timeout recovery.



Multi-agent swarm intelligence

Five specialized agents collaborate instead of one monolithic prompt.



Energy-aware scheduling

Tasks ordered by predicted productivity windows.



Wellness injection engine

Stretch, yoga, hydration, and recovery breaks, without hurting output.



Fault-tolerant AI inference

A defensive timeout wrapper prevents frontend blocking.



Live agent visualization

Staggered animations show each agent processing in sequence.



Voice daily briefing

Native browser speech synthesis narrates the schedule.

5-agent swarm workflow

User input flows sequentially through five specialized agents into a final, wellness-aware schedule.



Orchestrator agent

Coordinates the swarm end to end

1



Personalization agent

Energy analysis & user preferences

2



Task executor agent

Priority optimization & schedule build

3



Recommendation agent

Wellness injection & break optimization

4



Voice narrator agent

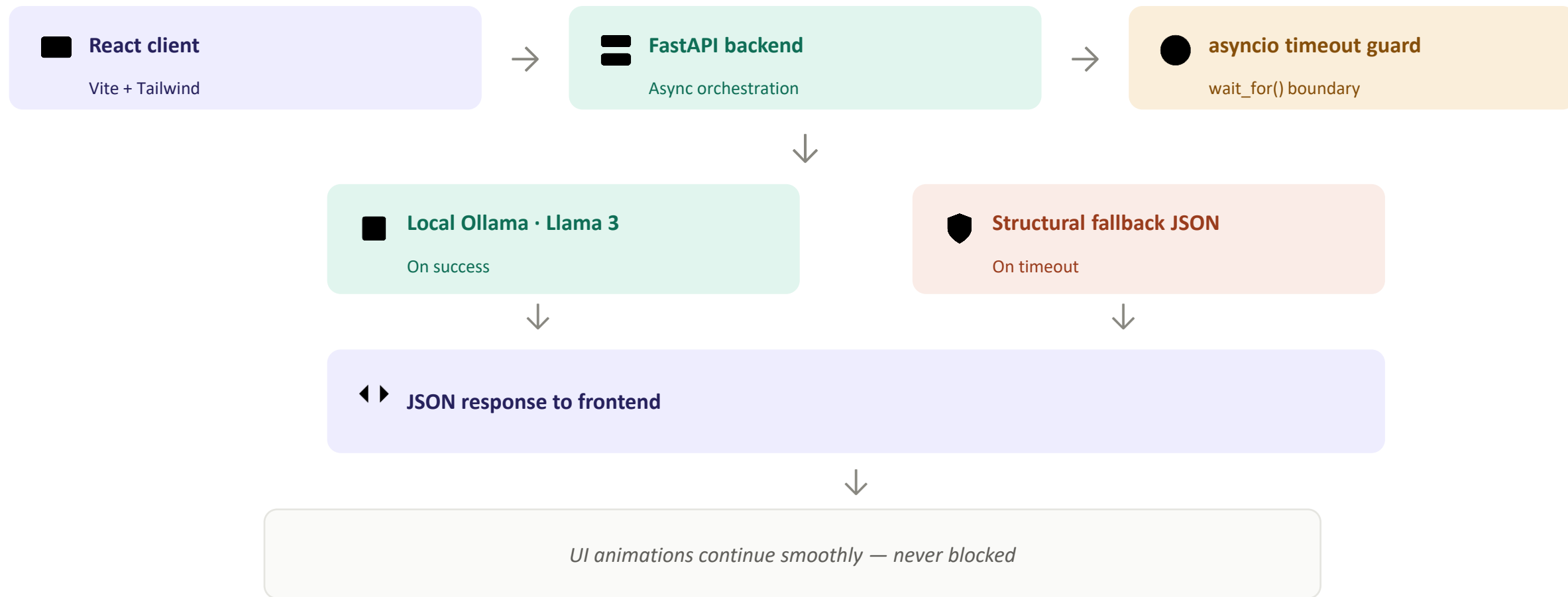
Daily summary & speech output

5

Final
schedule

System architecture

Async orchestration with a timeout guard sitting between the API and local model.





Defensive timeout strategy

Local LLM response time varies with hardware. Every inference call is wrapped in an `asyncio.wait_for()` boundary so the UI never freezes during live demos.



Model responds in time

LLM response → processed normally → full AI-generated schedule returned to the client.



Inference exceeds timeout

Timeout → fallback JSON returned → frontend animations continue uninterrupted.

Benefits

- Zero UI blocking
- Reliable live demos
- Predictable response contracts
- Graceful degradation
- Better UX on constrained hardware



Technology stack



Frontend

- React
- Vite
- Tailwind CSS
- Browser Speech Synthesis API



Backend

- FastAPI
- Python
- AsyncIO
- Pydantic
- Uvicorn



AI layer

- Ollama
- Llama 3



Dev tools

- Git
- GitHub
- VS Code



Future roadmap



Context awareness

Calendar sync, wearables, location & sleep-aware planning



Distributed swarm

Async DAG execution for concurrent agent reasoning



Persistent memory

PostgreSQL profiling & historical productivity analytics



Predictive wellness

Burnout detection & adaptive intervention scheduling

Engineering intelligent productivity through collaborative AI agents, resilient system design, and human-centric wellness optimization.

Team Seedhe Code · Smart Campus Hackathon 2026